

Infrathrone BattleOps: Week 2 Modules

MODULE 1: Control Plane Failures Under Load (API Server + etcd)

WHAT TO STUDY

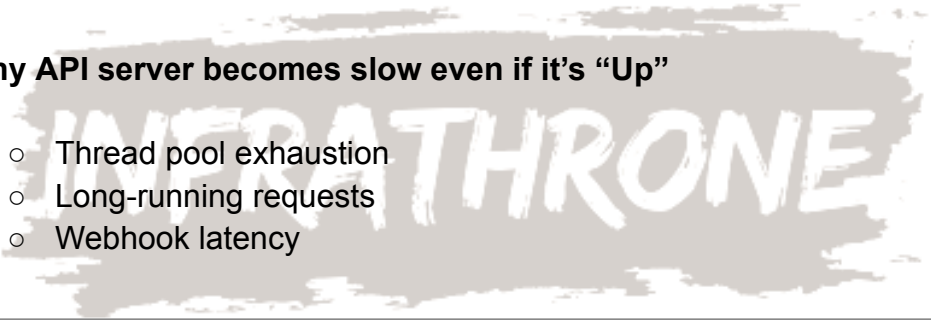
1. API Server internals

- Request flow → Authentication → Admission → etcd write

2. etcd failure modes

- High fsync latency
- Disk bottleneck
- Fragmentation
- Leader imbalance

3. Why API server becomes slow even if it's "Up"

- Thread pool exhaustion
 - Long-running requests
 - Webhook latency
- 
-

THE DEVOPS WAR ROOM

HANDS-ON

Step 1: Install a single-node control-plane

kubeadm init --pod-network-cidr=10.244.0.0/16

Apply CNI (Flannel): kubectl apply -f

<https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml>

Step 2: Check API Server health

- kubectl get --raw='/livez?verbose'
- kubectl get --raw='/readyz?verbose'

Step 3: Create controlled slowness

Introduce 500 dummy CRDs:

```
for i in {1..500}; do  
  kubectl apply -f https://k8s.io/examples/application/crd.yaml  
done
```

Step 4: Watch API server slow down

```
kubectl get pods -A --watch
```

You will see stuttering & late updates.

Step 5: Observe etcd latency

```
ETCDCTL_API=3 etcdctl endpoint status --write-out=table
```

WHERE TO STUDY

- API Server internals
<https://kubernetes.io/docs/concepts/overview/components/>
- etcd high latency debugging <https://etcd.io/docs/v3.5/faq/>

Infrathrone Pointers THE DEVOPS WAR ROOM

- 90% of “Kubernetes outage” starts with **etcd latency**
 - API Server is NOT “down”; it is saturated
 - CRDs and webhooks cause silent meltdowns
-

MODULE 2: Scheduler Deadlocks + Pod Eviction Logic

WHAT TO STUDY

1. How scheduler decides pod placement
 2. Impact of taints, tolerations & affinity
 3. Node Pressure conditions
 4. Why pods remain Pending even when nodes look Ready
-

HANDS-ON

Step 1: Create a tainted node

kubectl taint nodes <node> env=prod:NoSchedule

Step 2: Deploy a workload WITHOUT matching toleration

kubectl apply -f <https://k8s.io/examples/controllers/nginx-deployment.yaml>

Step 3: Debug Pending

kubectl describe pod <pod>

Step 4: Add toleration & see scheduling happen

tolerations:

- key: "env"

operator: "Equal"

value: "prod"

effect: "NoSchedule"

Step 5: Create resource deadlock

Deploy a pod with impossible resource demands:

resources:

requests:

cpu: "20"

memory: "50Gi"

WHERE TO STUDY

- Scheduler basics: <https://kubernetes.io/docs/concepts/scheduling-eviction/>
 - Taints/Tolerations:
<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>
-

Infrathrone Pointers

- Not all Pending is resource shortages; 40% caused by Taints
 - Scheduler is blind: it ONLY checks metadata, not workload runtime
 - If you can debug Pending fast → you are battle-ready
-



MODULE 3: Networking Chaos: kube-proxy, conntrack & DNS

WHAT TO STUDY

1. kube-proxy modes (iptables, IPVS)
 2. conntrack table exhaustion
 3. DNS caching, ndots behavior, retries
 4. When ping works but curl fails
-

HANDS-ON

Step 1: Check conntrack table

```
sudo conntrack -S
```

Step 2: Simulate DNS stress

```
for i in {1..5000}; do dig google.com & done
```

Step 3: Watch kube-dns crash loops

```
kubectl get pods -n kube-system -l k8s-app=kube-dns -w
```

Step 4: Debug DNS resolution vs TCP

```
dig google.com
```

```
curl -I https://google.com
```

Expected:

- dig works
- ping works
- curl fails → TCP path broken

Step 5: Restart CoreDNS cleanly

```
kubectl rollout restart deployment coredns -n kube-system
```

WHERE TO STUDY

- DNS in Kubernetes;
<https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>
 - conntrack debugging; <https://wiki.nftables.org/>
-

Infrathrone Pointers

- DNS failures, ≠ network down
 - ping + dig working but curl failing = Layer 4/5 exhaustion
 - conntrack is the silent killer of 20% Kubernetes outages
-

